

Probabilistic Machine Learning

Lecture 01: Introduction

Prof. Dr. Michael Granitzer

Chair of Data Science · University of Passau

Why Probability?

Guiding Question: Two models both say "fraud". What would you need to know before acting on either — and why does a label not tell you?

Machine Learning Gives Us Predictions

Three are flagged as fraud
and the three labels are identical

Café Central €4.20 legitimate	Supermarket €63.10 legitimate	Electronics Online €1,299.00 fraud	Petrol Station €78.40 legitimate
Gift Cards ×12 €600.00 fraud	Bookshop €24.90 legitimate	Hotel Booking €410.00 fraud	Pharmacy €12.75 legitimate

Machine Learning Gives Us Predictions

Three are flagged as fraud
and the three labels are identical

Café Central €4.20 legitimate	Supermarket €63.10 legitimate	Electronics Online €1,299.00 fraud	Petrol Station €78.40 legitimate
Gift Cards ×12 €600.00 fraud	Bookshop €24.90 legitimate	Hotel Booking €410.00 fraud	Pharmacy €12.75 legitimate

Are the three flagged transactions equally suspicious?

Machine Learning Gives Us Predictions

Three are flagged as fraud
and the three labels are identical

Café Central €4.20 legitimate	Supermarket €63.10 legitimate	Electronics Online €1,299.00 fraud	Petrol Station €78.40 legitimate
Gift Cards ×12 €600.00 fraud	Bookshop €24.90 legitimate	Hotel Booking €410.00 fraud	Pharmacy €12.75 legitimate

Are the three flagged transactions equally suspicious?

Would you block all three cards?

Machine Learning Gives Us Predictions

Three are flagged as fraud
and the three labels are identical

Café Central €4.20 legitimate	Supermarket €63.10 legitimate	Electronics Online €1,299.00 fraud	Petrol Station €78.40 legitimate
Gift Cards ×12 €600.00 fraud	Bookshop €24.90 legitimate	Hotel Booking €410.00 fraud	Pharmacy €12.75 legitimate

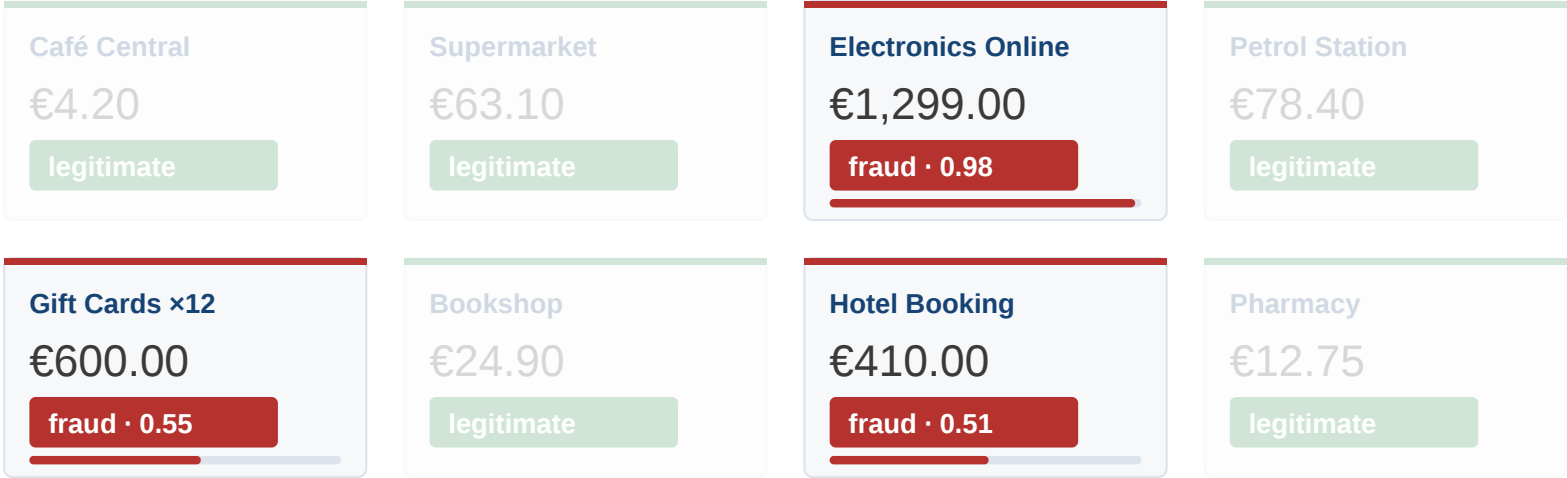
Are the three flagged transactions equally suspicious?

Would you block all three cards?

What would you need to know to treat them differently?

The Same Batch, With a Number Attached

Now every flagged case carries a number
0.98, 0.55, 0.51 — three very different cases



What can you do now that you could not do a minute ago?

What a Number Lets Us Do

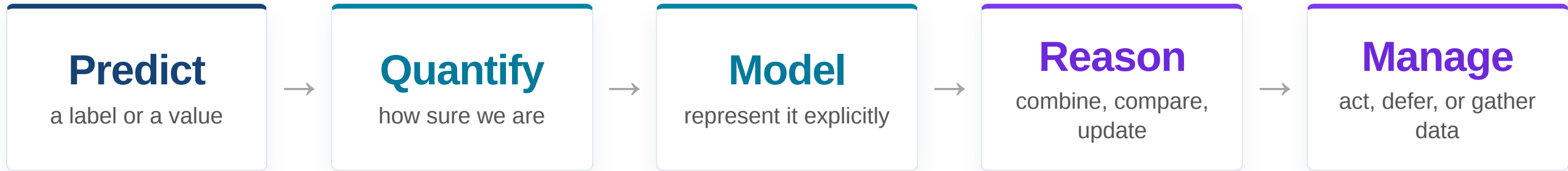
The number lets us	What that means for this batch
Compare and rank	separate the €1,299 order from the two borderline cases — and compare two models that would both just say "fraud"
Weigh consequences	set the threshold from the cost of a missed fraud against the cost of an annoyed customer
Defer deliberately	send 0.55 and 0.51 to a human ask whether more data would move them
Combine and communicate	merge evidence from several signals report <i>how sure</i> we are, not only what we think

What a Number Lets Us Do

The number lets us	What that means for this batch
Compare and rank	separate the €1,299 order from the two borderline cases — and compare two models that would both just say "fraud"
Weigh consequences	set the threshold from the cost of a missed fraud against the cost of an annoyed customer
Defer deliberately	send 0.55 and 0.51 to a human ask whether more data would move them
Combine and communicate	merge evidence from several signals report <i>how sure</i> we are, not only what we think

We need to model uncertainty — and we need to be able to reason with it.

From Prediction to Managed Uncertainty



And once we want to manage it — **where does the uncertainty actually come from?**

Probabilistic machine learning is not a different way of producing predictions. It is a **language for reasoning about predictions that are not certain.**

Mini-Exercise

A loan model gives three applicants a **default probability** of **0.05**, **0.35** and **0.45**.

COST OF ACCEPTING A DEFAULTER

€8,000

the loan is not repaid

COST OF REJECTING A GOOD APPLICANT

€500

the margin we would have earned

1. Which applicants does the rule "reject if $p > 0.5$ " reject?
2. **Weigh each option by how likely its bad outcome is.** What is the expected cost of *accepting* each applicant, and of *rejecting* each?
3. At which probability do the two expected costs balance?

Mini-Exercise — Solution

Mini-Exercise — Solution

Weigh each outcome by its probability: $\mathbb{E}[\text{accept}] = p \cdot 8,000$ against $\mathbb{E}[\text{reject}] = (1 - p) \cdot 500$.

	p	Accept	Reject	Cheaper by	Decision
A	0.05	€400	€475	1.2×	accept — <i>only just</i>
B	0.35	€2,800	€325	8.6×	reject, clearly
C	0.45	€3,600	€275	13×	reject, overwhelmingly

Break-even: $p \cdot 8,000 = (1 - p) \cdot 500 \Rightarrow p^* = \frac{500}{8,500} \approx 0.059$



What the Threshold Tells Us

THE INTUITIVE RULE

reject if $p > 0.5$

rejects **nobody** here — every applicant is below it, including one whose default risk is nine times the bank's break-even

THE RULE THE COSTS IMPLY

reject if $p > 0.059$

because a default costs **16×** what a lost customer does

The threshold comes from the **costs**, not from the probability. 0.5 is only the right cut-off when both mistakes are equally expensive — which they almost never are.

- **B and C are not close calls:** rejecting is 8.6× and 13× cheaper.
- **A is the only borderline case** — €400 against €475, and $p = 0.05$ sits just under $p^* = 0.059$. This is where a second signal is worth buying.

Takeaway

A prediction without a measure of its uncertainty cannot be compared, weighed against consequences, or handed to a human at the right moment. **Modelling uncertainty and reasoning with it** is what turns a prediction into a decision.

Where Uncertainty Comes From

Guiding Question: The model says 0.55. Is that because the world is noisy, or because the model has not learned enough — and would more data help?

Two Reasons to Be Unsure

The model says 0.55. Before asking how to *represent* that, ask why it is not 0.99.

Two Reasons to Be Unsure

The model says 0.55. Before asking how to *represent* that, ask why it is not 0.99.

Is the transaction itself ambiguous — would a perfect model, with all the data in the world, still be unsure?

Two Reasons to Be Unsure

The model says 0.55. Before asking how to *represent* that, ask why it is not 0.99.

Is the transaction itself ambiguous — would a perfect model, with all the data in the world, still be unsure?

Or has the model simply not seen enough cases like this one?

Two Reasons to Be Unsure

The model says 0.55. Before asking how to *represent* that, ask why it is not 0.99.

Is the transaction itself ambiguous — would a perfect model, with all the data in the world, still be unsure?

Or has the model simply not seen enough cases like this one?

Would collecting more data help — and could *any* amount of data remove the doubt entirely?

Two Examples

EXAMPLE A · AN AMBIGUOUS OBSERVATION

Two patients with **identical** records — same age, same symptoms, same test results. One recovers, one does not.

✗ Would more patients like these help?
Never.

EXAMPLE B · AN UNFAMILIAR CASE

A house-price model trained on flats of 40–120 m² is asked about a **600 m² villa**. It has no examples anywhere near it.

✓ Would ten villas in the training data help?
Completely.

Which uncertainty can be reduced, and which is part of the problem itself?

The Distinction — Then the Names

	Where it comes from	More data?	Our example
Uncertainty in the problem	the outcome genuinely varies given what we observe	✗ never	identical patients, different outcomes
Uncertainty in our knowledge	we have not seen enough to pin the model down	✓ yes	the 600 m ² villa

The Distinction — Then the Names

	Where it comes from	More data?	Our example
Uncertainty in the problem	the outcome genuinely varies given what we observe	✗ never	identical patients, different outcomes
Uncertainty in our knowledge	we have not seen enough to pin the model down	✓ yes	the 600 m ² villa

Aleatoric

from alea, dice — noise in the world

Epistemic

from episteme, knowledge — ignorance in the model

Why the Distinction Matters

If the uncertainty is...	The right response	The expensive mistake
in our knowledge (epistemic)	collect data where the model is thin defer to a human who knows more	accepting it as inherent and never improving
in the problem (aleatoric)	model the noise set a policy for the ambiguous cases	buying more data that cannot help

A model that can tell the two apart can report that it is **out of its depth**, not merely unsure.

A probabilistic model can represent **both** kinds of uncertainty — and can often tell you which one it is facing. That is why the distinction is worth a name.

Takeaway

Uncertainty has two sources: **noise in the world** and **gaps in our knowledge**. Only the second shrinks with data, and a decision — collect more, defer, act — depends on knowing which one you face.

Beyond True or False

Guiding Question: A proof either holds or it does not. A learned prediction is never a proof — so what kind of conclusion is it, and how strong is it?

Three Modes of Reasoning

	From	To	What it guarantees
Deduction <i>(logic)</i>	premises	what must follow	certain — but adds no new content
Induction <i>(statistics)</i>	observed patterns	a regularity , a hypothesis	nothing — content-adding, not guaranteed
Abduction <i>(probability)</i>	observations	the best available explanation	nothing — only comparable against rivals

Machine learning lives in the bottom two rows.



The Scientific Reasoning Cycle

The three modes are not alternatives — they take turns.

Example: gravity.

Deduction

From the theory: all objects of equal mass fall at the same rate everywhere on earth.

Induction

Design an experiment, measure fall times at several locations, compute means and standard deviations.

Abduction

Times agree at one location but vary between locations.
Best explanation: distance from earth's mass centre.

The cycle turns: the abducted explanation becomes the next theory to deduce from. A data-science project runs the same loop, usually several times.

Mini-Exercise

A dashboard shows that customers who use the mobile app churn **less** than those who do not.

- 1 What would a purely **inductive** reading conclude?
- 2 Give two different **abductive** explanations.
- 3 What **deduction** would let you distinguish them?

Mini-Exercise

A dashboard shows that customers who use the mobile app churn **less** than those who do not.

- 1 What would a purely **inductive** reading conclude?
- 2 Give two different **abductive** explanations.
- 3 What **deduction** would let you distinguish them?

INDUCTION

"App use reduces churn" — a regularity read straight off the data.

ABDUCTION

The app causes loyalty — *or* loyal customers install the app. Same correlation, opposite causal direction.

DEDUCTION

If the app causes loyalty, randomly prompting a subgroup to install it should lower their churn — an experiment deduced from the hypothesis.

Beyond True or False

	Classical reasoning	Probabilistic reasoning
Conclusions	follow, or do not	are supported to <i>degrees</i>
Evidence	premises, taken as given	may be partial or noisy
Certainty	the conclusion is certain	the conclusion may stay uncertain
New information	can only add premises	can <i>change</i> what we believe

Classical logic asks *whether* a conclusion follows. Probabilistic reasoning also asks **how strongly the evidence supports each possible conclusion**.

What the Language Has to Do

We now know that uncertainty matters and that it has different sources. What we still lack is a **systematic way** to:

- 1 **Represent** — say how much we believe each possibility
- 2 **Combine** — put several uncertain pieces of information together
- 3 **Update** — change our beliefs when new evidence arrives
- 4 **Decide** — act on beliefs that are not certainties

What the Language Has to Do

We now know that uncertainty matters and that it has different sources. What we still lack is a **systematic way** to:

- 1 **Represent** — say how much we believe each possibility
- 2 **Combine** — put several uncertain pieces of information together
- 3 **Update** — change our beliefs when new evidence arrives
- 4 **Decide** — act on beliefs that are not certainties

Probability provides that language. The formal material that follows exists because each of these four operations turns out to have exactly one consistent set of rules.

Takeaway

Machine-learning predictions are inferences from evidence, not derivations from premises.
Probability is the calculus of such inferences — which is why the next lecture is about it, and why it will feel like an answer rather than a prerequisite.

What Probabilistic ML Enables

Guiding Question: Which questions about a machine-learning system will you be able to answer after this course that you cannot answer today?

After This Course, You Will Be Able to Ask — and Answer

ABOUT A PREDICTION

- ? How certain is it, and should I trust it?
- ? When should the model refuse to decide?
- ? What alternatives are plausible?

ABOUT THE DATA

- ? When would more data help?
- ? When is uncertainty unavoidable?
- ? What does this data actually justify?

ABOUT LEARNING

- ? How should belief change when evidence arrives?
- ? How do I combine prior knowledge with observations?
- ? How do I compare competing explanations?

ABOUT THE METHODS

- ? Why do the common methods work, and under which assumptions?
- ? When should I expect them to fail?
- ? What is my model assuming without telling me?

What Changes Compared to Traditional Machine Learning

TRADITIONAL ML ASKS

What is the prediction?

PROBABILISTIC ML ALSO ASKS

- ? **How certain** is it?
- ? **What alternatives** are plausible?
- ? **Why** are we uncertain?
- ? **How does new evidence** change what we believe?
- ? **What decision** follows from this?
- ? **What can and cannot** be inferred from this information?

Takeaway

Probabilistic machine learning does not replace prediction. It **surrounds every prediction with the questions a decision needs answered** — and gives one language in which all of them can be asked.

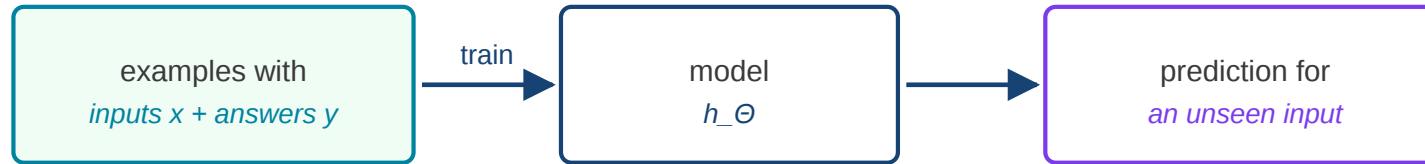
How Machines Learn

Guiding Question: Nobody writes the rule. So what exactly is a machine given, what does it build, and where does uncertainty get in?

Learning From Answers

A radiologist has labelled ten thousand scans. A bank has ten years of loans and knows which ones defaulted. In both cases **every example carries its answer**.

Supervised: every training example carries the answer



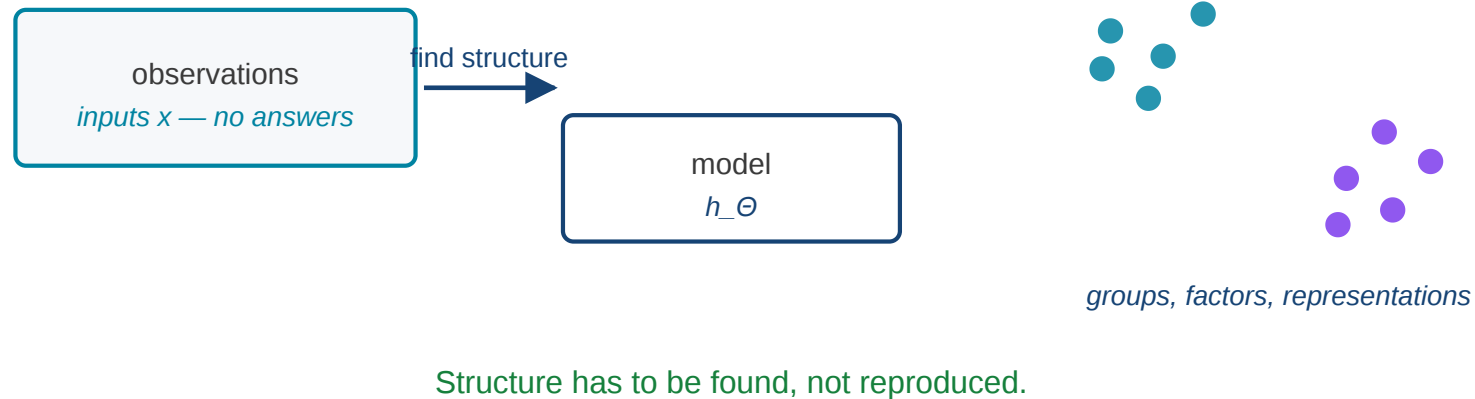
The learner is told the correct output for each case it sees.

What information does the learner receive during training?

Learning Without Answers

Now the same bank wants to know **what kinds of customer it has**. Nobody has ever written down the list — the categories are what we are looking for.

Unsupervised: no answers — only the observations themselves

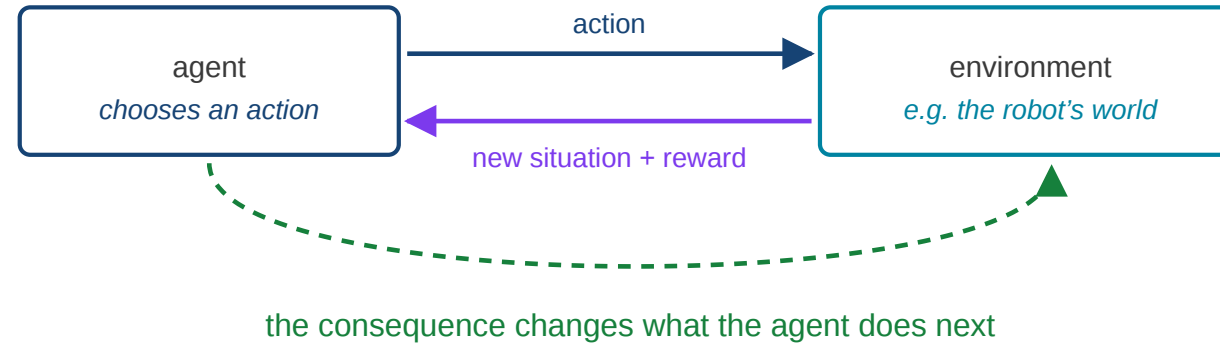


What can we learn when nobody tells us the correct answer?

Learning by Acting

A robot has to cross a room. Nobody can label the correct motor command for every situation it might find itself in — and it only discovers whether a move was good **after making it**.

Reinforcement: no answer is given — only the consequence of acting



The learner must act to find out whether the action was good.

How can a learner improve when it must act in order to find out whether the action was any good?

Same Goal, Different Information

Same goal, different information

	WHAT IT IS GIVEN	WHAT FEEDBACK IT GETS
Supervised	inputs + answers	the answer, every time
Unsupervised	inputs only	nothing
Reinforcement	the situation it acts in	a reward, after acting

Same Goal, Different Information

Same goal, different information

	WHAT IT IS GIVEN	WHAT FEEDBACK IT GETS
Supervised	inputs + answers	the answer, every time
Unsupervised	inputs only	nothing
Reinforcement	the situation it acts in	a reward, after acting

The paradigms differ in **what the learner is given** and **what feedback it gets** — not in what we ultimately want from it.



What Is Common to All Three?

Nobody wrote the rule

The mapping from situation to answer is **too complex or too implicit** to specify by hand.

Examples exist

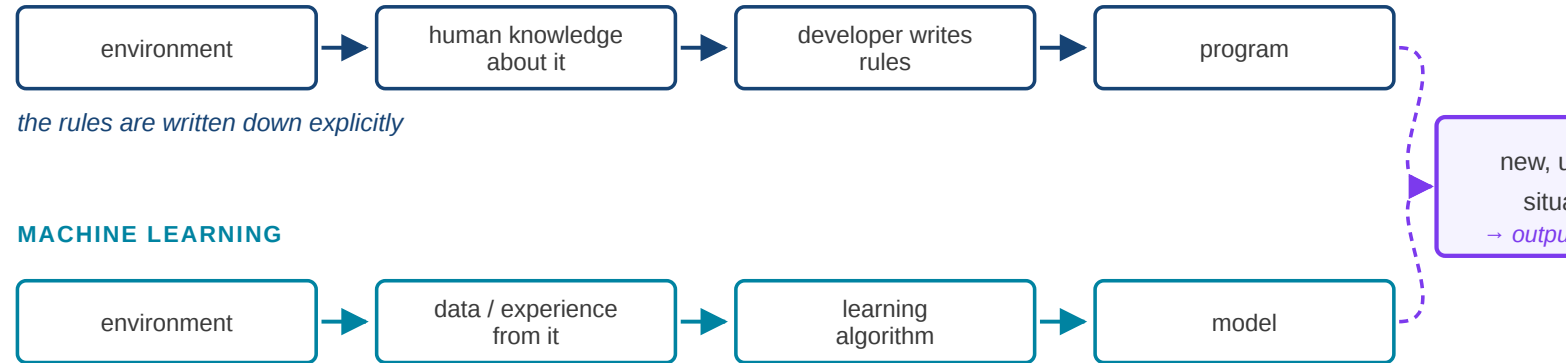
There is data — or experience — in which the desired behaviour is demonstrated, even if it was never described.

So what replaces the rule that nobody wrote?

Two Ways to Get Behaviour

Both are judged on situations neither of them has seen

TRADITIONAL PROGRAMMING



the rules are written down explicitly

MACHINE LEARNING



the rules are inferred from what the environment produced

What Actually Changed

TRADITIONAL PROGRAMMING

The developer's **knowledge becomes rules**. Deduction: rules plus input give the output.

The assumptions are visible — they are in the code.

MACHINE LEARNING

The environment's **data becomes a model**. Induction: examples plus a learning procedure give a rule.

The assumptions are still there — but now they are in the data, the model family and the objective.

The rules do not disappear when nobody writes them. They become **harder to inspect** — which is why the rest of this lecture is about making every part of that pipeline explicit.

What Would It Mean for a Machine to Learn?

Before any definition — three questions you would have to answer about *any* system before you could say it had learned anything:

- ? What is it supposed to get better at?
- ? What does it get to look at while getting better?
- ? How would we know whether it improved?

What Would It Mean for a Machine to Learn?

Before any definition — three questions you would have to answer about *any* system before you could say it had learned anything:

- ? What is it supposed to get better at?
- ? What does it get to look at while getting better?
- ? How would we know whether it improved?

Task T

what the system should learn to do

Experience E

what it learns from

Performance P

how we decide it improved

Chess



A program that plays, loses, and plays again — millions of times against itself.

- **Task** — play chess well
- **Experience** — games played against itself, and positions from expert play
- **Performance** — fraction of games won; rating against other engines

A Large Language Model



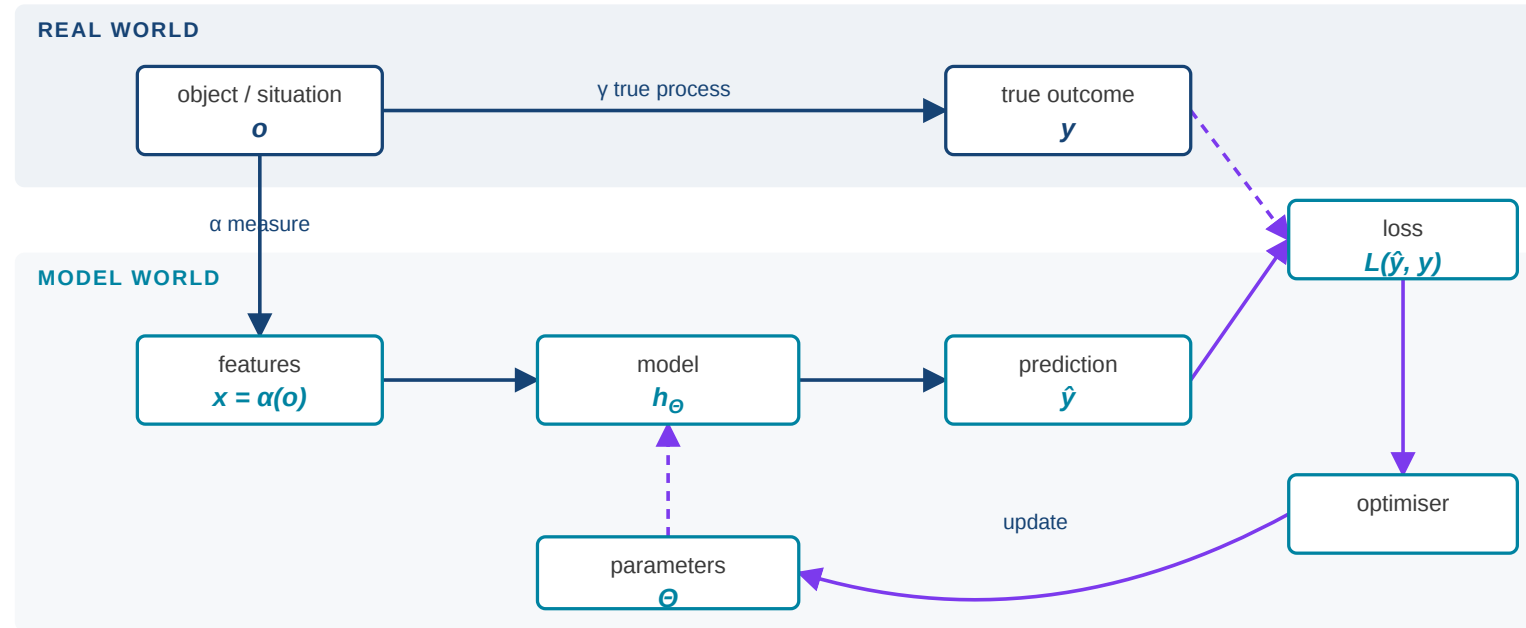
A model shown enormous quantities of text, asked only to continue it.

- **Task** — predict the next word
- **Experience** — a very large corpus of text, then human feedback
- **Performance** — likelihood of held-out text; later, human judgement

Two systems with nothing in common — a board game and a text corpus — fit the **same three-part description**. That is what makes it a definition rather than a description of one program.

What Would We Ideally Like to Know?

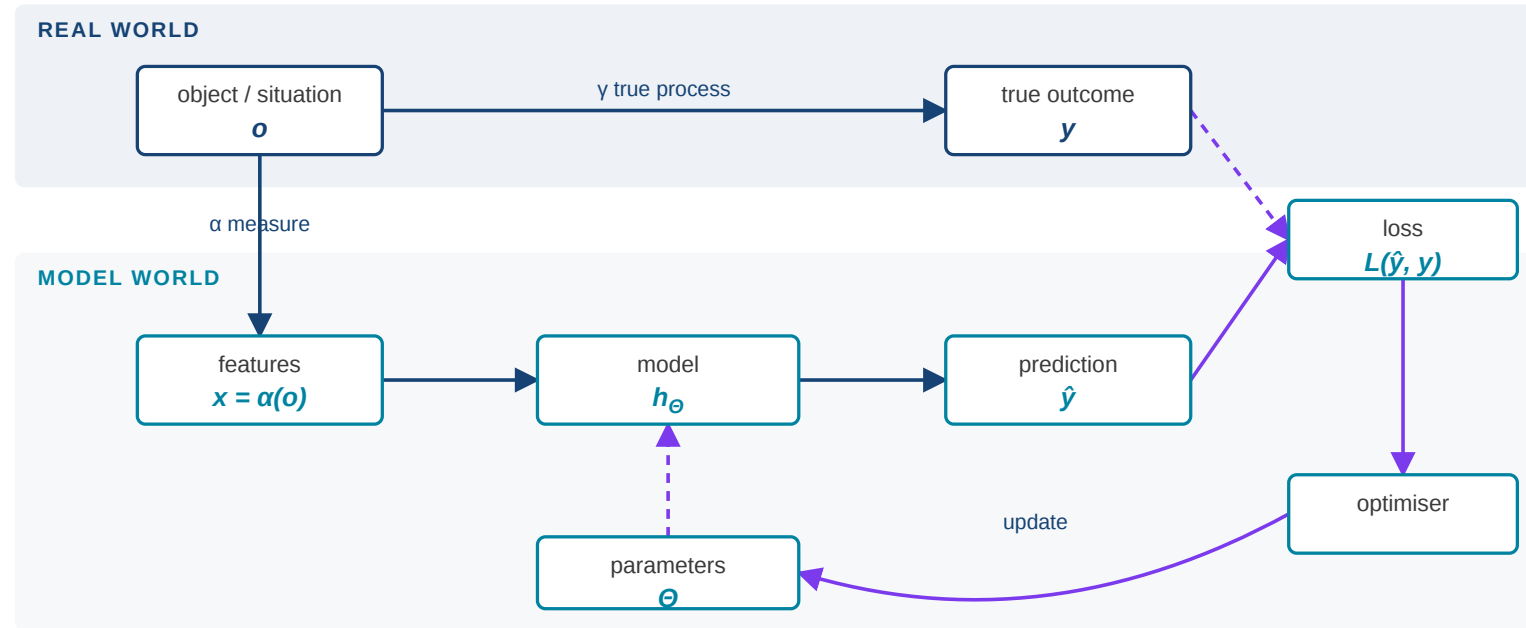
The optimiser changes the parameters — and the training loop closes



There is some real process that turns a situation into an outcome. Why can we not simply write it down?

From the Real World to a Model of It

The optimiser changes the parameters — and the training loop closes



Two Worlds, One Correspondence

$$\gamma(o); \rightsquigarrow; h_{\Theta}(\alpha(o)); \equiv; h_{\Theta}(x)$$

γ

the **real process** — unknown,
and not ours to write down

α

the **representation** — what
we chose to measure

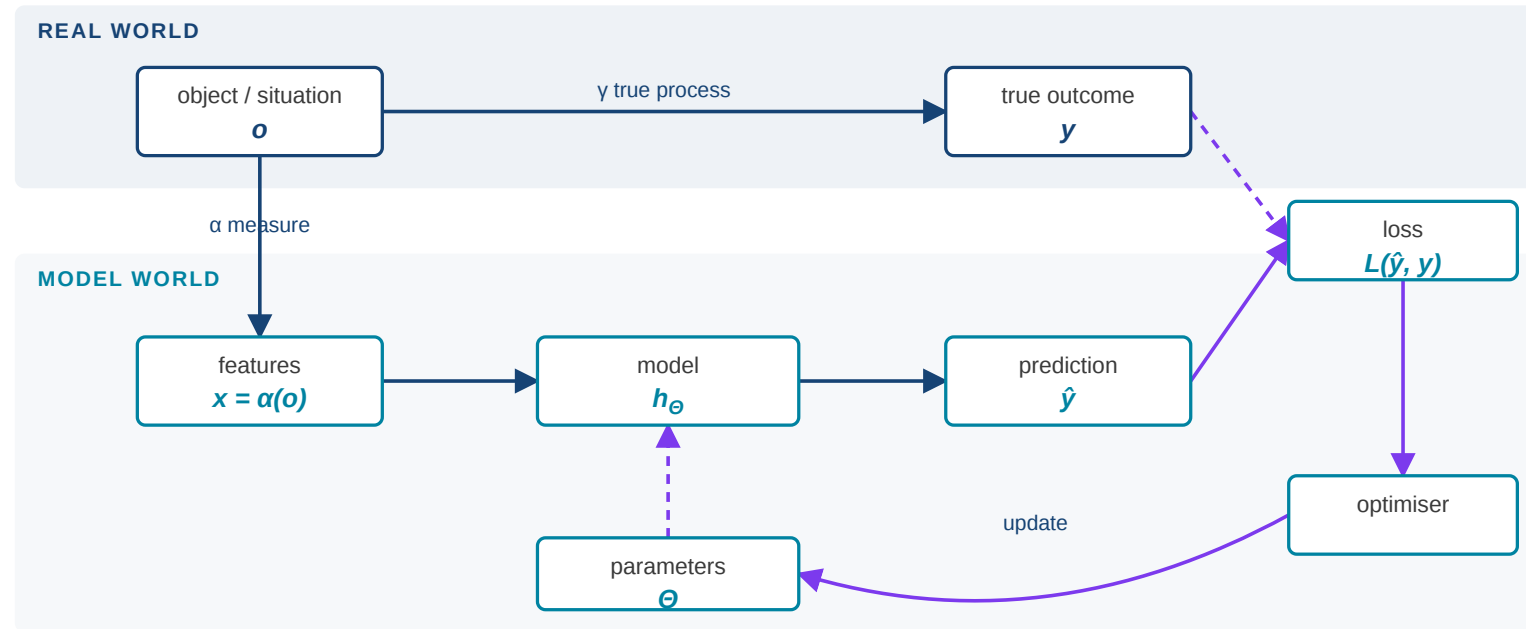
h_{Θ}

the **model** — our
approximation, with
parameters Θ

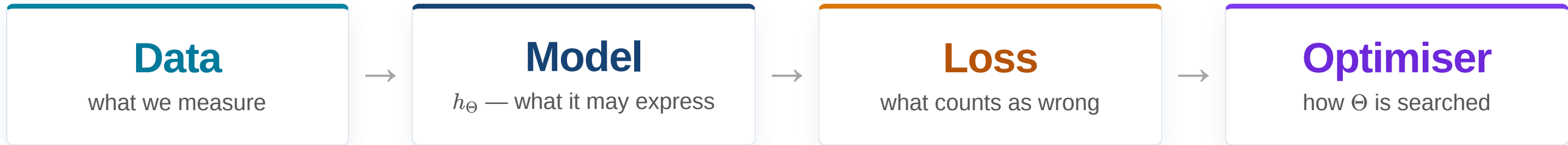
We never learn the world. We build a **model world** and approximate the process inside it.

Closing the Loop

The optimiser changes the parameters — and the training loop closes



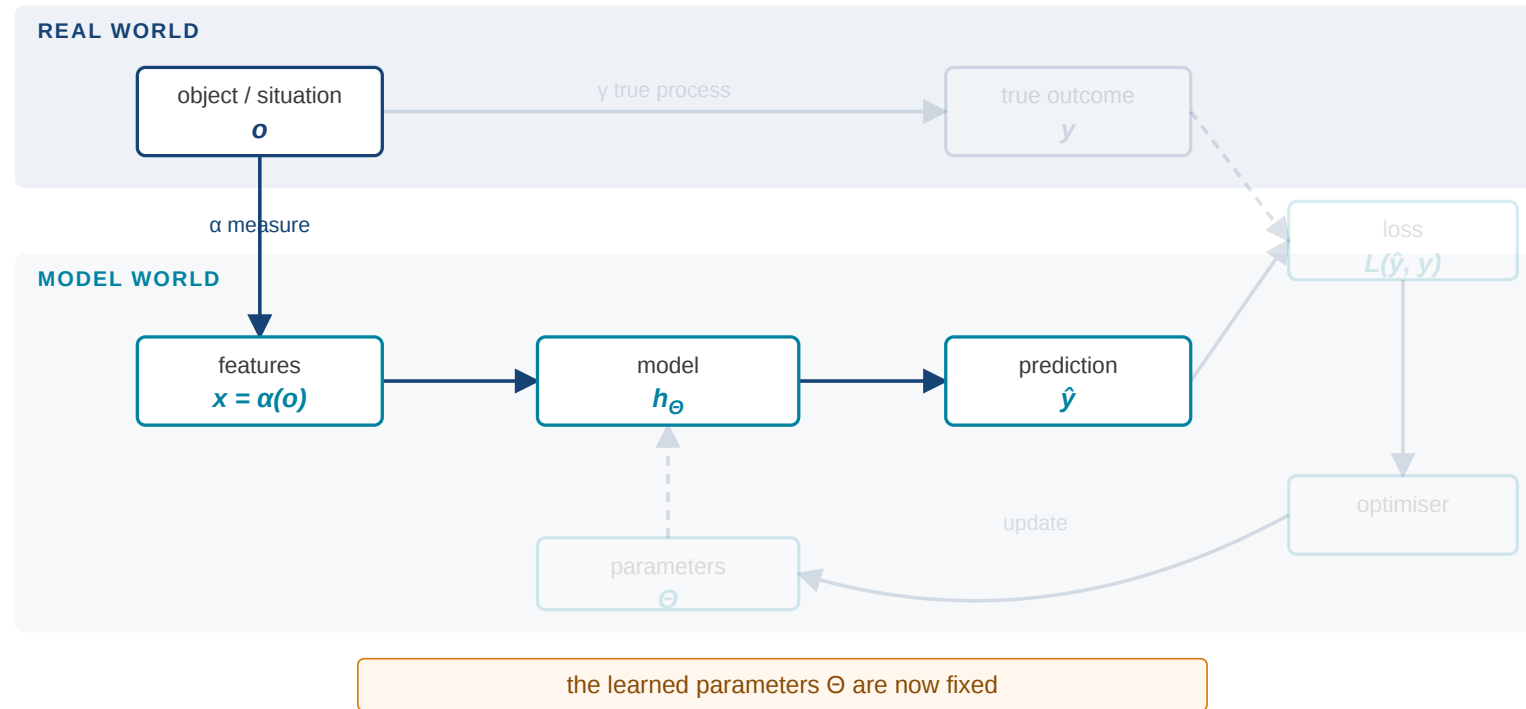
The Four Choices



The optimiser has settings of its own that are not learned from the data — **hyperparameters**. Choosing them is part of the outer process, not of the loop.

Training Is Not Using

Inference: a new, unseen situation follows one path — the training machinery is gone



Which of these boxes still exist when the system is actually deployed?

The Loop Around the Loop

Data, model, loss and optimiser describe **what runs**. They say nothing about how anybody decided on them.

INNER LOOP · THE LEARNING SYSTEM

data → model → prediction → loss →
optimiser → updated parameters

*runs thousands of times per minute,
automatically*

OUTER LOOP · THE DEVELOPMENT PROCESS

collect → represent → split → train →
validate → diagnose → revise → evaluate
→ deploy → monitor

*runs a handful of times per project, and
every step is a human decision*

What the Outer Loop Decides

BEFORE TRAINING

- what to collect, and from where
- how to clean and represent it
- how to split train / validation / test

WHILE DEVELOPING

- which model family, which hyperparameters
- which errors to look at, and why they happen
- what to change next

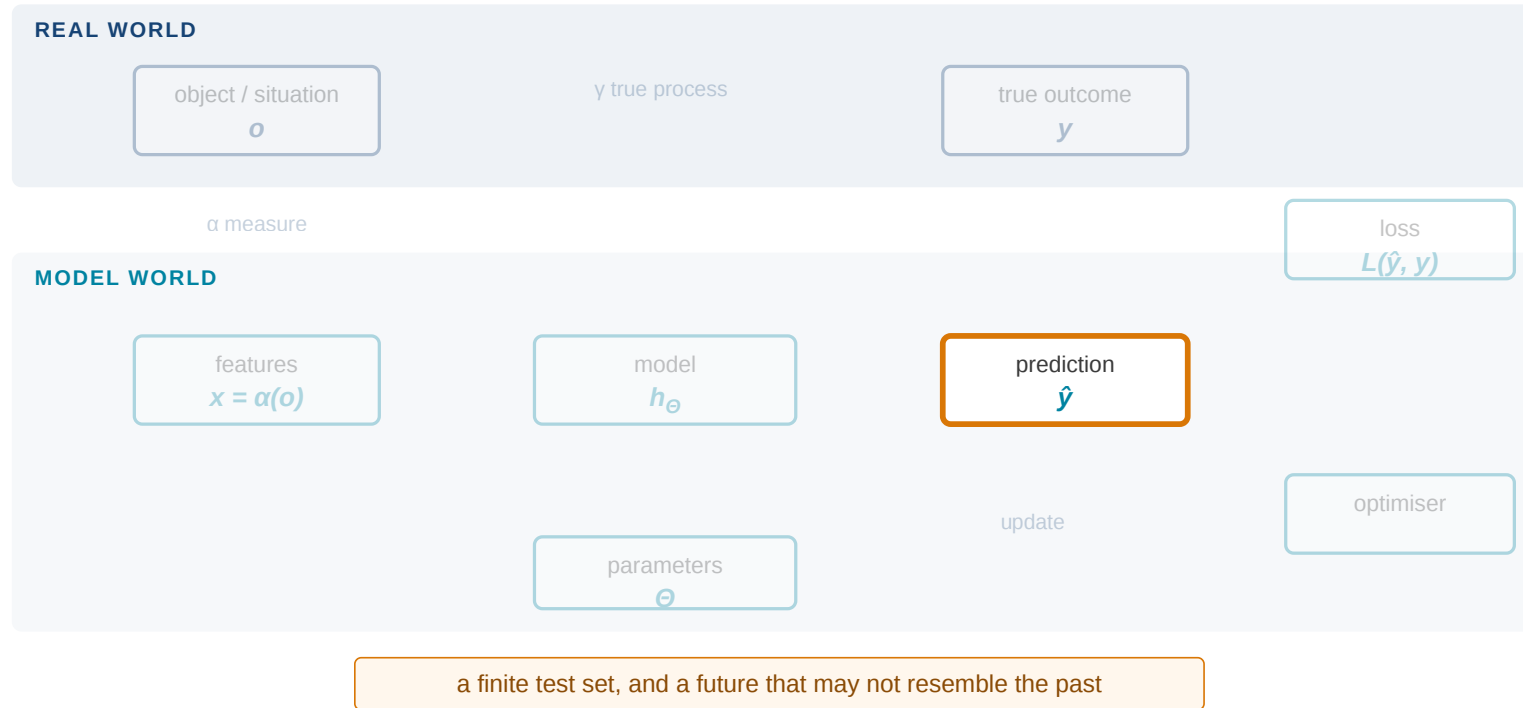
AFTER TRAINING

- is it good enough, and by whose measure
- does it hold up on data it has never seen
- is it still holding up next month

Machine learning is **experimental**: you make choices, then test whether they worked. Measured performance is *evidence* about future performance — never a guarantee of it.

The Same System, Read for Uncertainty

In the evaluation



Two Distinctions Worth Keeping Straight

UNCERTAINTY *IN* THE LOSS

The quantities entering the loss are uncertain: y is noisy, the sample is finite. The loss itself is an ordinary function.

UNCERTAINTY *ABOUT* THE LOSS

Whether this objective represents the task at all. That is a modelling judgement, not a random variable.

Do not say "the loss is random". Say what is uncertain: the data it is computed from, or our confidence that it is the right objective.

Two Distinctions Worth Keeping Straight

UNCERTAINTY *IN* THE LOSS

The quantities entering the loss are uncertain: y is noisy, the sample is finite. The loss itself is an ordinary function.

UNCERTAINTY *ABOUT* THE LOSS

Whether this objective represents the task at all. That is a modelling judgement, not a random variable.

Do not say "the loss is random". Say what is uncertain: the data it is computed from, or our confidence that it is the right objective.

Uncertainty is not a property of the final prediction. It **enters throughout the system** — and probabilistic machine learning is the language for representing and reasoning about it at every one of these places.

Designing the Learning Problem

Before any model is chosen, four things have to be decided — and each of them is a claim about the world.

THE TASK

- ? What should the system predict or decide?
- ? What is one unit of prediction?
- ? What counts as success?

THE DATA

- ? Which observations represent the problem?
- ? How were the examples sampled?
- ? Will future cases look like these?

THE REPRESENTATION A

THE TARGET



? What are the inputs, concretely?

? Which features are available — and which are merely convenient?

? What is lost in the measurement?

? What is γ , in words a domain expert would accept?

? Is the label we have the thing we care about, or a proxy for it?

Designing the System

MODEL AND OBJECTIVE

- ? What does the model family assume, and how flexible is it?
- ? Does minimising this loss correspond to what we actually want?

LEARNING

- ? Which procedure searches the parameters?
- ? How sensitive is the result to initialisation and randomness?

Designing the System — Evaluation

EVALUATION AND GENERALISATION

- ? Which metric matters, and which data is held out?
- ? How uncertain is the estimate of performance?
- ? What happens when the environment changes?

UNCERTAINTY AND USE

- ? Which uncertainty must be represented explicitly?
- ? Is the prediction informing a decision — and what do the errors cost?

Designing the System — Evaluation

EVALUATION AND GENERALISATION

- ? Which metric matters, and which data is held out?
- ? How uncertain is the estimate of performance?
- ? What happens when the environment changes?

UNCERTAINTY AND USE

- ? Which uncertainty must be represented explicitly?
- ? Is the prediction informing a decision — and what do the errors cost?

These are the questions the rest of the course answers. Each lecture takes one or two of them and makes them precise.

The Road Ahead

Guiding Question: Now that the motivation is in place — which lecture answers which of your questions?

The Road Ahead

Each lecture answers one of the questions you have just met.

02

Representing uncertainty *How do we describe uncertainty mathematically?*

Probability, distributions, and how to summarise data.

03

Learning from evidence *Why do the classical methods work — and what do they assume?*

Linear and logistic regression, re-read as probabilistic models.

04

Combining information *How do uncertain quantities relate — and when can we treat them separately?*

Naive Bayes, independence, and the graphical view.

05

Structured models *How do we write down a large, structured set of beliefs?*

Bayesian networks, fitting them, and drawing them.

06

Judging models *What is the quantity we actually want to minimise — and what did we get instead?*

Evaluation and empirical risk minimisation.

07

Deciding and refining *How do we keep uncertainty all the way to the decision?*

Mixtures, Bayesian regression, and what comes after.

The Programme, in Three Moves

Every method in this course is the same three moves, in the language you are about to learn:

Model the joint

Write down what the world would have to be like to produce what we observed — the data *and* the unknowns together.

Condition on the data

Inference is conditioning: fix what was observed, and let the rules of probability say what follows for the rest.

Read off answers

Predictions, uncertainties and decisions are all quantities computed from that one conditioned model.

Every one of these words — joint, condition, inference — gets a precise meaning in Lecture 02. They appear here only so that you recognise them when they return.